

Agent Wiki: Evidence-Closed Local-First Markdown Knowledge Bases for LLM Agents

Zijun Chu

Independent Researcher

<https://github.com/NimaChu/agent-wiki>

Journal Manuscript Draft, July 2026

Abstract

Large language model (LLM) agents increasingly perform multi-step software and knowledge-work tasks, but their durable knowledge often remains trapped in chat transcripts, proprietary note systems, hidden memories, or retrieval indexes that are difficult for users to inspect. This article presents Agent Wiki, a local-first, Markdown-native knowledge substrate for LLM coding agents and human users who need source-grounded, auditable, and low-cost long-term knowledge management. Agent Wiki separates raw evidence notes from synthesized wiki pages, treats ordinary wiki links as a computable graph, and introduces an evidence-closure invariant for processed sources: a source can be considered processed only when its related wiki targets resolve, the wiki layer links back to the raw evidence, and explicit follow-up flags are closed. The system also indexes visual evidence by extracting, mirroring, and inventorying source images, and it provides an optional read-only dashboard for graph inspection and maintenance queues. We describe the design, implementation, and reproducibility workflow, then evaluate the artifact through a single-corpus case study on a technical documentation vault. On 2026-07-09, the vault contained 1,120 raw source notes, 113 wiki pages, and 5,353 graph edges; local lint checks reported zero unresolved links, zero orphaned wiki pages, and zero processed-source closure violations. Agent Wiki does not replace retrieval-augmented generation; instead, it provides an inspectable evidence and synthesis layer from which retrieval, dashboards, and agent workflows can be built.

Keywords: LLM agents; software engineering; local-first software; Markdown; knowledge management; evidence-grounded AI; personal knowledge bases; reproducible tools.

1 Introduction

LLM agents are moving from short-form assistance toward long-running workflows: reading documentation, editing source code, maintaining project context, answering from private corpora, and coordinating multiple local tools. These workflows require durable knowledge. A useful agent should not repeatedly rediscover the same documentation, lose the provenance of a claim, or collapse the distinction between raw evidence and synthesized understanding. Yet many practical agent memory systems still rely on transient chat history, application-specific memories, opaque vector indexes, or human-maintained note vaults that were not designed for automated maintenance.

This creates a gap for independent developers and small teams. They often want the benefits of an agent-maintained knowledge base without deploying a hosted database, paying for a specialized knowledge platform, or accepting an opaque memory layer that cannot be reviewed with ordinary developer tools. They also need stronger provenance than a flat folder of summaries. When an answer depends on source material, the system should make it possible to inspect the original capture, the synthesized wiki page, and the links between them.

Agent Wiki addresses this gap with a deliberately conservative design: local files first, Markdown first, and evidence first. It is a repository layout, a set of agent-operating rules, a command line interface, templates, and an optional dashboard for maintaining a source-grounded local knowledge base. The raw layer stores source captures and metadata. The wiki layer stores durable knowledge units, such as concepts, methods, APIs, workflows, entities, comparisons, and recurring questions. Local scripts scan the Markdown files, resolve links, build graph statistics, report maintenance queues, and enforce a processed-source closure rule.

The guiding premise is that agent knowledge should be inspectable before it is automated. Agent Wiki therefore treats the graph as a derived artifact rather than the source of truth. A user can inspect, diff, search, copy, version, publish, or delete the underlying files without needing the dashboard or a database server. This local-first posture follows the principle that users should retain ownership of their data even when cloud services or agents are used around it [1].

This article makes four contributions:

1. It presents a file-native architecture for agent-maintained knowledge bases that separates raw evidence from synthesized wiki knowledge.
2. It formalizes an evidence-closure invariant for processed source notes and implements local checks for unresolved links, orphaned wiki pages, invalid relations, and processed-gate failures.
3. It introduces an image-evidence workflow that mirrors and inventories visual source material so screenshots, diagrams, and UI states can remain available as local evidence.
4. It reports a case-study evaluation over a technical documentation vault, showing that local health checks can preserve graph consistency across more than one thousand raw source notes.

2 Background and Motivation

2.1 Agent Workflows Need External Memory

LLMs are increasingly used as controllers that interleave reasoning, tool use, and environment feedback. ReAct showed that language models can combine reasoning traces with actions to solve interactive tasks [2]. Software-engineering agents extend this idea to repositories, issue trackers, tests, terminals, and code search. Benchmarks and systems such as SWE-bench and SWE-agent demonstrate that code-oriented agents can attempt real software tasks when equipped with appropriate interfaces [3, 4].

As these workflows lengthen, memory becomes a practical systems problem. A coding agent may need to remember local conventions, documentation facts, decisions, evidence, screenshots, and unresolved follow-ups. Keeping this knowledge only in chat history is fragile. Storing it only in an embedding index improves retrieval but can make provenance and maintenance harder to inspect. Storing it only in a human note-taking application may help the human but leaves the agent without a disciplined operating model.

2.2 RAG Is Complementary but Not Sufficient

Retrieval-augmented generation (RAG) connects a language model to external documents at answer time and has become a standard pattern for knowledge-intensive generation [5]. However, RAG does not by itself specify how source captures should be preserved, how synthesized concepts should evolve, how evidence links should be audited, or how an agent should decide

that a source has been fully processed. RAG optimizes retrieval; Agent Wiki optimizes the preparation and maintenance of the corpus that retrieval may later consume.

This distinction is important for small local projects. A user may want a knowledge base that can be read without a model, maintained with ordinary text tools, and later embedded or indexed if needed. Agent Wiki is therefore not an alternative to RAG in the sense of answer-time retrieval. It is a staging and audit layer for source-grounded knowledge.

2.3 Personal Knowledge Tools Are Not Agent Protocols

Markdown note systems such as Obsidian and Logseq show the value of local files, backlinks, and graph-oriented thinking [8, 9]. Evergreen note practices emphasize durable, atomic notes that become more useful through linking and revision [7]. Agent Wiki adopts these habits but narrows them for agentic operation. It defines a repository structure, templates, status fields, closure rules, and scripts that an LLM coding agent can execute consistently.

The difference is procedural rather than cosmetic. Agent Wiki is not merely a folder of Markdown files. It specifies what counts as raw evidence, what counts as synthesized wiki knowledge, when a source may be marked processed, which links must resolve, which relation hints are supported, and which dashboard artifacts are read-only.

3 Design Goals

Agent Wiki is shaped by six design goals.

Local ownership. The user should own source captures, synthesized pages, image inventories, snapshots, and maintenance state as local files. Public project code and private knowledge can be separated by Git policy.

Agent-native operation. The repository should be legible to coding agents. The project includes workspace rules, templates, and root npm scripts so agents can capture sources, search, maintain, lint, repair links, and answer from the knowledge base without depending on globally installed skills.

Evidence before synthesis. Source captures should remain available after synthesis. The wiki layer is a compiled knowledge layer, not a replacement for evidence.

Low operational cost. The system should not require a hosted database, a paid subscription, an embedding service, an always-on dashboard, or a separate editor runtime.

Auditable maintenance. Maintenance state should be computable from files. Broken links, orphaned wiki pages, unresolved related targets, stale sources, follow-up flags, and processed-gate failures should be visible to local commands.

Incremental extensibility. The file-native substrate should not prevent future indexing, dashboards, retrieval tools, or structured agent APIs. Derived artifacts should be rebuildable from Markdown.

4 System Overview

Agent Wiki is organized as a self-contained repository. Table 1 summarizes the major directories.

Directory	Role
<code>raw/</code>	Source notes, snapshots, evidence metadata, and image inventories.
<code>wiki/</code>	Durable synthesized knowledge pages.
<code>templates/</code>	Reusable raw-source and wiki-page templates.
<code>src/</code>	Canonical command line source for capture, search, lint, repair, and graph generation.
<code>scripts/</code>	Backwards-compatible command launchers and optional local helpers.
<code>tools/wiki-dashboard/</code>	Optional read-only graph dashboard.

Table 1: Agent Wiki repository layout.

4.1 Raw Evidence Layer

The raw layer is the factual source layer. A raw note stores bibliographic and capture metadata such as title, author, date, URL, captured date, type, status, snapshot path, content hash, capture method, source quality, tags, related wiki targets, and follow-up state. The body can preserve captured text, extracted claims, image inventories, candidate wiki links, and processing notes.

Raw notes are intentionally not treated as disposable summaries. A single raw source may update many wiki pages, and a single wiki page may link back to many raw sources. This design prevents a source from being collapsed into one summary page when it contains multiple reusable ideas.

4.2 Wiki Layer

The wiki layer is the compiled knowledge layer. It favors one durable knowledge unit per page: a concept, entity, API, method, workflow, comparison, or recurring question. Pages use Obsidian-compatible links such as `[[Page Name]]` and `[[Page Name|label]]`. Navigation files such as `wiki/index.md` and `wiki/log.md` are treated as maintenance records rather than graph knowledge nodes.

Wiki pages may include typed relation hints from a deliberately small vocabulary: `supports`, `challenges`, `related_to`, `applies_to`, `company_of`, and `product_of`. The small set keeps relations easy to validate and avoids turning the vault into an uncontrolled ontology project.

4.3 Command Line Interface

The root command line interface exposes local workflows through npm scripts. The main entry point delegates to specialized scripts for capture, status, lint, garden review, link repair, search, image indexing, IMA import, and dashboard generation. Table 2 lists the primary commands.

5 Evidence-Closure Model

The key invariant in Agent Wiki is evidence closure for processed raw sources. Let R be the set of raw source notes and W the set of wiki pages. For a raw source $r \in R$, let $\text{related}(r)$ be the list of wiki targets declared in the source metadata, and let $\text{resolve}(x)$ map a wiki-link target to an existing node or to failure. Let $\text{links}(w)$ be the set of links from a wiki page w .

A processed source is considered closed only if:

Command	Purpose
<code>npm run wiki:status</code>	Report vault graph and queue statistics.
<code>npm run wiki:lint</code>	Validate unresolved links, metadata, and processed-source closure.
<code>npm run wiki:garden</code>	Show maintenance queues and weakly connected pages.
<code>npm run wiki:repair-links</code>	Assist link repair.
<code>npm run wiki:search - "query"</code>	Search titles, tags, aliases, and excerpts.
<code>npm run wiki:capture</code>	Create raw source notes.
<code>npm run wiki:images</code>	Extract, mirror, and index image evidence.
<code>npm run dashboard:open</code>	Refresh graph, run the local dashboard, and open the browser.

Table 2: Primary local workflows exposed as root npm scripts.

$$\begin{aligned}
\text{processed}(r) \Rightarrow & |\text{related}(r)| > 0 \\
& \wedge \forall t \in \text{related}(r), \text{resolve}(t) \neq \perp \\
& \wedge \exists w \in W, w \in \text{resolve}(\text{related}(r)) \wedge r \in \text{links}(w) \\
& \wedge \neg \text{needsFollowup}(r).
\end{aligned}$$

This invariant is intentionally modest. It does not prove that every claim in a wiki page is correct. It does not prove that the source has been exhaustively processed. It does, however, prevent a common agent-maintenance failure: a source is marked done but is unreachable from the durable wiki layer, or the wiki layer has no backlink to the evidence that supposedly supports it.

The lint workflow reports processed-source issues using four reason codes: **missing-related**, **unresolved-related**, **missing-wiki-backlink**, and **explicit-followup**. This turns a subjective maintenance label into a reviewable local property.

6 Image Evidence

Many sources contain visual evidence: UI states, screenshots, diagrams, configuration panels, charts, and workflows. Ordinary text-only capture can lose this evidence or leave it dependent on remote image URLs. Agent Wiki therefore includes a dedicated image workflow.

The image script scans a raw note and selected snapshots for Markdown image syntax and HTML `img` tags. It resolves relative URLs against the source URL, deduplicates images, mirrors remote images into `raw/assets/<source-note>/`, records local paths, content type, byte counts, source context, and download errors, writes a machine-readable `image-index.json`, and updates the raw note’s `Images` section with a compact inventory table.

Agents are instructed to promote only useful visual evidence into wiki pages. The exhaustive inventory remains in the raw layer. This keeps answers concise while preserving the complete visual record when a user needs to audit a screenshot or diagram.

7 Graph Generation and Dashboard

The scanner walks Markdown files under `raw/`, `wiki/`, `templates/`, and archives. It parses simple frontmatter, extracts body links and frontmatter links, resolves targets by id, title, basename, and alias, and emits nodes, edges, unresolved links, typed relations, invalid relations, incoming counts, outgoing counts, and node excerpts.

The dashboard is a read-only visualization surface. It displays global and local graph views, a wiki-only knowledge view by default, evidence drill-down for one wiki page and its directly linked raw sources, search, group and status filters, selected-node links and backlinks, tags, raw/wiki/link counts, and health indicators for inbox, follow-up, stale, broken-link, and processed-gate queues.

The dashboard intentionally does not write to the vault. Write operations remain explicit Markdown edits or command line workflows, which makes the graph a derived artifact rather than a hidden source of truth.

8 Implementation

Agent Wiki is implemented with Node.js scripts and a Vite/React dashboard. The public software artifact is small and inspectable: the root package exposes the core scripts, and the dashboard is an optional local frontend. The system has no required database server, no required cloud backend, and no required Obsidian runtime.

The main scanner is implemented in `src/wiki-lib.mjs`. It provides frontmatter parsing, wiki-link extraction, link resolution, relation parsing, vault scanning, graph statistics, and processed-source issue detection. The command dispatcher in `src/karpathy-wiki.mjs` maps user-facing npm scripts to implementation modules. Image indexing is implemented in `src/image-assets.mjs`. Dashboard generation is implemented under `tools/wiki-dashboard/`.

The implementation chooses simple structured parsing over ad hoc shell pipelines. This matters because the same rules are used by human users, coding agents, lint commands, and dashboard generation. A link that resolves in the dashboard should resolve in lint and search as well.

9 Evaluation Method

The evaluation is a case-study evaluation of the artifact as a local maintenance substrate. The goal is not to show that Agent Wiki improves model answer quality over all RAG systems. Instead, the evaluation asks whether the system can maintain inspectable graph health over a documentation-scale vault using local commands.

We ask three research questions:

RQ1: Can the vault maintain link and evidence-closure health over more than one thousand source notes?

RQ2: Can health be checked by local, reproducible commands rather than by manual dashboard inspection?

RQ3: What practical limitations remain for journal-level claims and future evaluation?

The case-study corpus is a local technical documentation vault centered on Autodesk FlexSim documentation. The vault was maintained using Agent Wiki workflows and checked on 2026-07-09 with:

```
npm run wiki:status
npm run wiki:lint
```

The commands report graph statistics, unresolved links, invalid relations, processed-source issues, orphaned wiki pages, weak wiki pages, missing claim sources, and missing metadata fields.

Metric	Value
Total nodes	1,244
Graph edges	5,353
Raw source notes	1,120
Wiki pages	113
Pending raw sources	0
Inbox raw sources	0
IMA pointer notes	0
Processed raw sources	1,119
Needs follow-up	0
Stale sources	0
Unresolved links	0
Invalid relation hints	0
Orphaned wiki pages	0
Processed-source closure issues	0

Table 3: Case-study vault health reported by Agent Wiki local commands on 2026-07-09.

10 Case-Study Results

Table 3 reports the vault state on 2026-07-09. The local vault contained 1,244 graph nodes, 5,353 graph edges, 1,120 raw source notes, and 113 wiki pages. Lint reported no unresolved links, invalid relation hints, processed-source issues, orphaned wiki pages, weak wiki pages, missing claim sources, missing frontmatter, missing status fields, or missing type fields.

These results answer RQ1 and RQ2 for the case-study setting. The vault remained mechanically healthy under the implemented checks despite containing more than one thousand raw source notes. The absence of processed-source closure issues is particularly important: it indicates that processed source notes had resolved related wiki targets and corresponding wiki backlinks, rather than being isolated summaries.

The result should be interpreted as a systems-health result, not an answer- quality benchmark. Link health and closure health are necessary but not sufficient for factual correctness. They provide an auditable substrate on which answer quality can later be evaluated.

11 Discussion

11.1 Why Markdown First?

Markdown gives Agent Wiki a low-friction shared representation for humans, agents, Git, search tools, and dashboards. It can be edited in a terminal or a graphical editor, diffed in Git, indexed by scripts, and copied without export steps. This does not make Markdown semantically rich by itself. Agent Wiki adds structure through conventions: frontmatter fields, source statuses, supported relation hints, raw/wiki separation, and closure checks.

11.2 Why Not Start with Embeddings?

Embeddings are useful for retrieval, but they are a derived index. Starting with an embedding database can hide source quality, duplicate captures, weak synthesis, broken backlinks, and unresolved follow-ups. Agent Wiki starts from inspectable source and synthesis files. An embedding index can be added later from either layer.

11.3 What Agents Gain

An agent using Agent Wiki gains operational handles rather than only documents. It can search the vault, inspect the index, read source-backed wiki pages, detect broken links, process inbox items, repair evidence closure, and include local visual evidence in answers. The repository itself tells the agent how to behave, which reduces reliance on hidden application memory.

11.4 What Users Gain

Users gain a private knowledge base whose maintenance state can be audited without trusting the agent. If an answer cites a wiki page, the user can inspect that page. If the page cites raw evidence, the user can inspect the source note, snapshot, or image inventory. If the vault claims a source is processed, the user can run the same lint checks.

12 Threats to Validity

Single-corpus evaluation. The case study uses one local technical documentation corpus. The result may not generalize to highly dynamic web sources, collaborative writing, legal materials, scientific PDFs, or multimodal corpora with large binary assets.

Single-developer workflow. The current artifact was developed and maintained primarily by one independent developer with agent assistance. Multi-user editing, merge conflicts, role-based review, and permission models require further study.

Health is not correctness. The reported metrics measure graph consistency and closure invariants. They do not prove that every wiki claim faithfully summarizes its sources or that agent answers are factually superior to RAG baselines.

Public artifact versus local corpus. The software artifact is public, but private or bulky raw knowledge can remain local by design. This improves privacy but means that some case-study vault content may require regeneration from public sources or a separately published snapshot for independent replication.

Simple parsing model. The current frontmatter parser and link resolver intentionally cover a simple Markdown subset. This improves portability but may not support every convention used by larger static-site generators or note-taking systems.

13 Future Work

Future work falls into five areas. First, Agent Wiki could expose a Model Context Protocol server so agents can query vault state through structured APIs while preserving Markdown as the source of truth. Second, optional embedding indexes could be generated from raw notes, wiki pages, or both, enabling comparative evaluation of graph-guided search and vector retrieval. Third, evidence closure could move from page-level backlinks to claim-level citation checks. Fourth, image evidence could be enriched with OCR, diagram captions, and visual similarity search. Fifth, future studies should compare answer quality, maintenance effort, and user trust against chat-only memory, note-only workflows, and RAG-first baselines.

14 Conclusion

Agent Wiki shows that an LLM-agent memory substrate can begin as a local Markdown repository rather than a hosted service or hidden vector database. By separating raw evidence from synthesized wiki knowledge, enforcing processed- source closure, indexing images as evidence, and deriving graph health from plain files, the system offers a practical bridge between personal knowledge management, source-grounded agent workflows, and future retrieval pipelines. The case-study vault demonstrates that the approach can maintain local graph health across more than one thousand source notes. The broader claim is modest but important: before agent knowledge becomes more automated, it should become more inspectable.

Availability

The Agent Wiki software artifact is available at <https://github.com/NimaChu/agent-wiki>. The archived software release used for this manuscript is available through Zenodo at <https://doi.org/10.5281/zenodo.21366895>. The manuscript draft was prepared against repository release v0.2.1 and local vault status checked on 2026-07-15. The public release contains the reusable software, paper materials, and a metadata-only FlexSim 2026 case-study subset; private or bulky raw knowledge vaults can be kept local by design.

CRedit Author Statement

Zijun Chu: Conceptualization, methodology, software, validation, investigation, writing–original draft, writing–review and editing, visualization, project administration.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Acknowledgements

The author thanks the open-source communities whose tools and documentation make local-first software development practical.

References

- [1] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, pages 154–178, 2019.
- [2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. <https://arxiv.org/abs/2210.03629>.

- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. <https://arxiv.org/abs/2310.06770>.
- [4] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv preprint, 2024. <https://arxiv.org/abs/2405.15793>.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. <https://arxiv.org/abs/2005.11401>.
- [6] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv preprint, 2023. <https://arxiv.org/abs/2310.08560>.
- [7] Andy Matuschak. Evergreen notes. <https://notes.andymatuschak.org/>.
- [8] Obsidian. Obsidian. <https://obsidian.md/>.
- [9] Logseq. Logseq. <https://logseq.com/>.
- [10] Khoj AI. Khoj documentation. <https://docs.khoj.dev/>.